6BUIS018W – Information Driven Entrepreneurship and Enterprise





Lecture 4

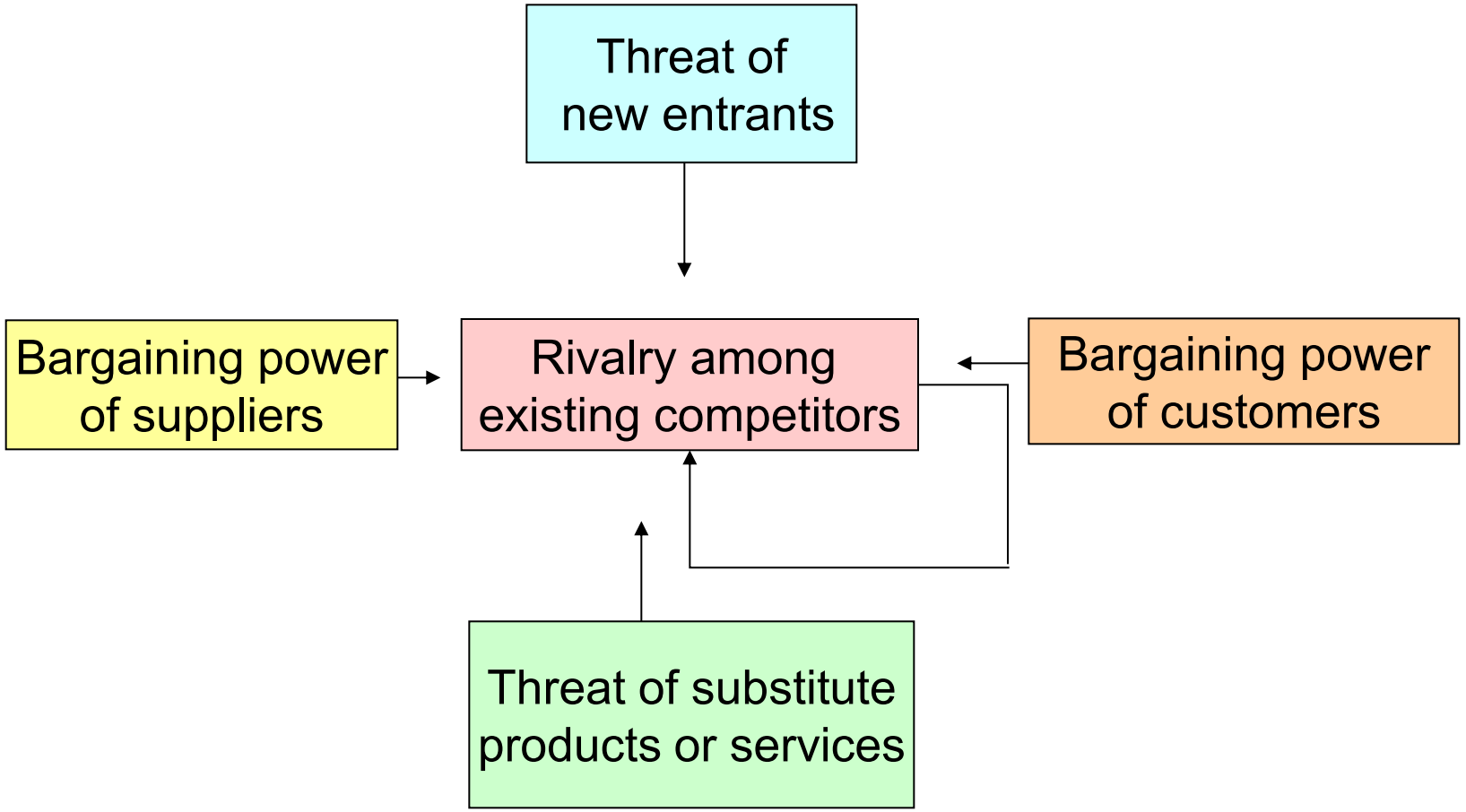
E-commerce Design

Prof Tamas Kiss

Previously…

Porter's 5 forces model

to position yourself on the market and define strategies



Previously…

SWOT matrix analysis

EXTERNAL FACTORS \ INTERNAL FACTORS	Strengths (S)	Weaknesses (W)
Opportunities (O)	**SO Strategies** Generate strategies here that use strengths to take advantages of opportunities	**WO Strategies** Generate strategies here that take advantage of opportunities by overcome weaknesses
Threats (T)	**ST Strategies** Generate strategies here that use strengths to avoid threats	**WT Strategies** Generate strategies here that minimize weaknesses and avoid threats

Prof Tamas Kiss

Previously…

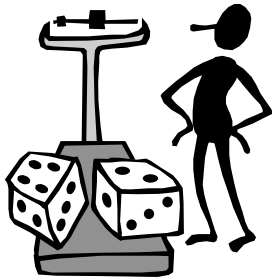
CSF analysis

Critical Success Factors (CSF) method - John Rockart 1979

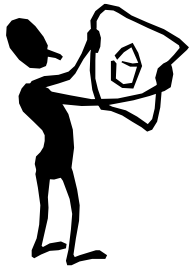
1. **Defining business objectives** (SMART objectives)
2. **Defining key factors that ensure business success**
   - **CSFs:**
     - Those limited number of areas where "things must go right“



3. **Defining measures** of successful achievement of CSF's

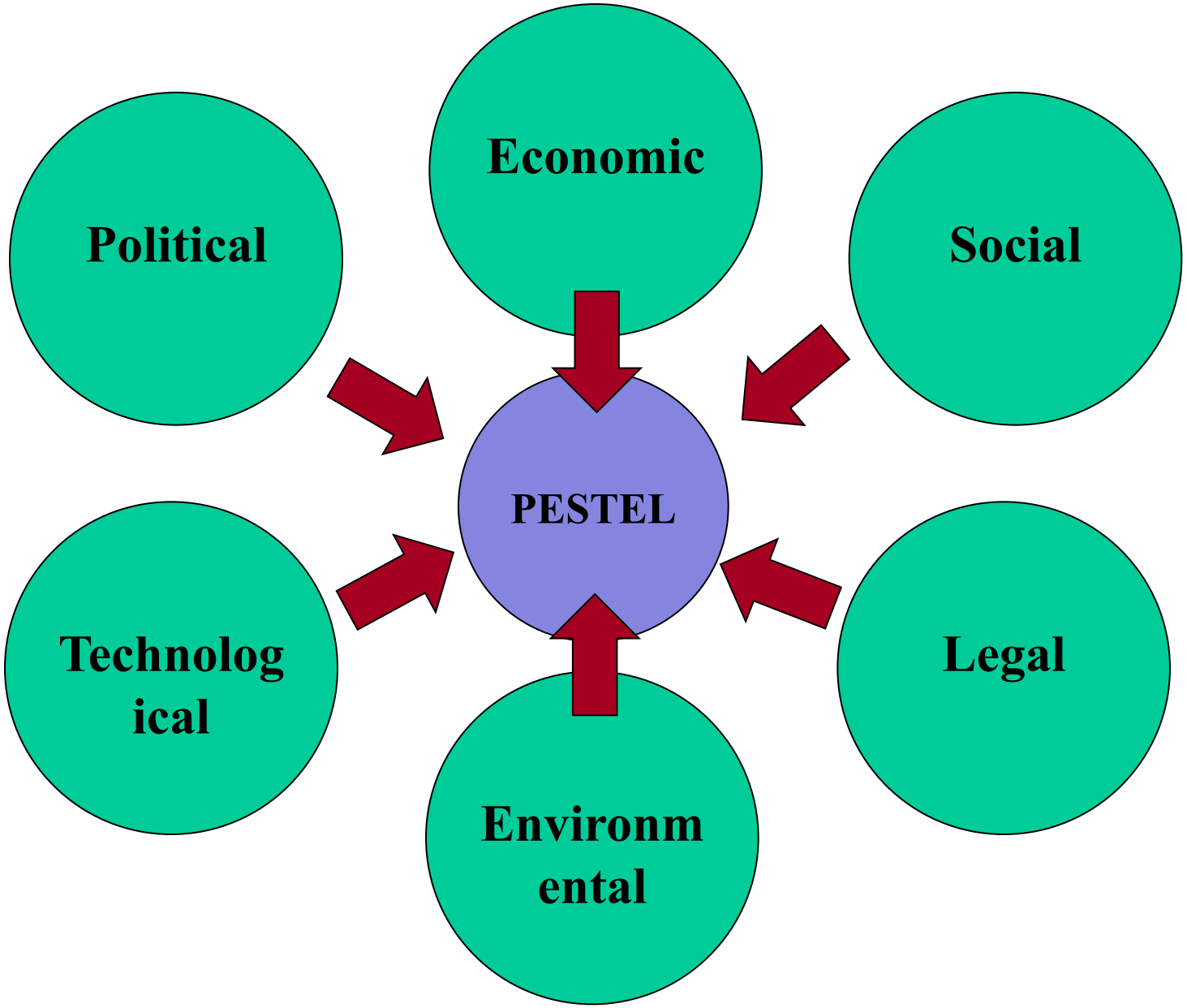


4. **Information system** should facilitate **reporting** on CSF



Analysis of the macro environment

PESTLE (PESTEL) analysis



E-business systems analysis and design

Further reading:

Dave Chaffey, Tanya Hemphill, and David Edmundson-Bird: **Digital Business and E-Commerce Management,** ***7th ed. Pearson Education, 2019.***

Essential: Chapter 11 – Analysis and design

Further:

Chapter 10 – Change management

Chapter 12 – Digital business service implementation and optimization

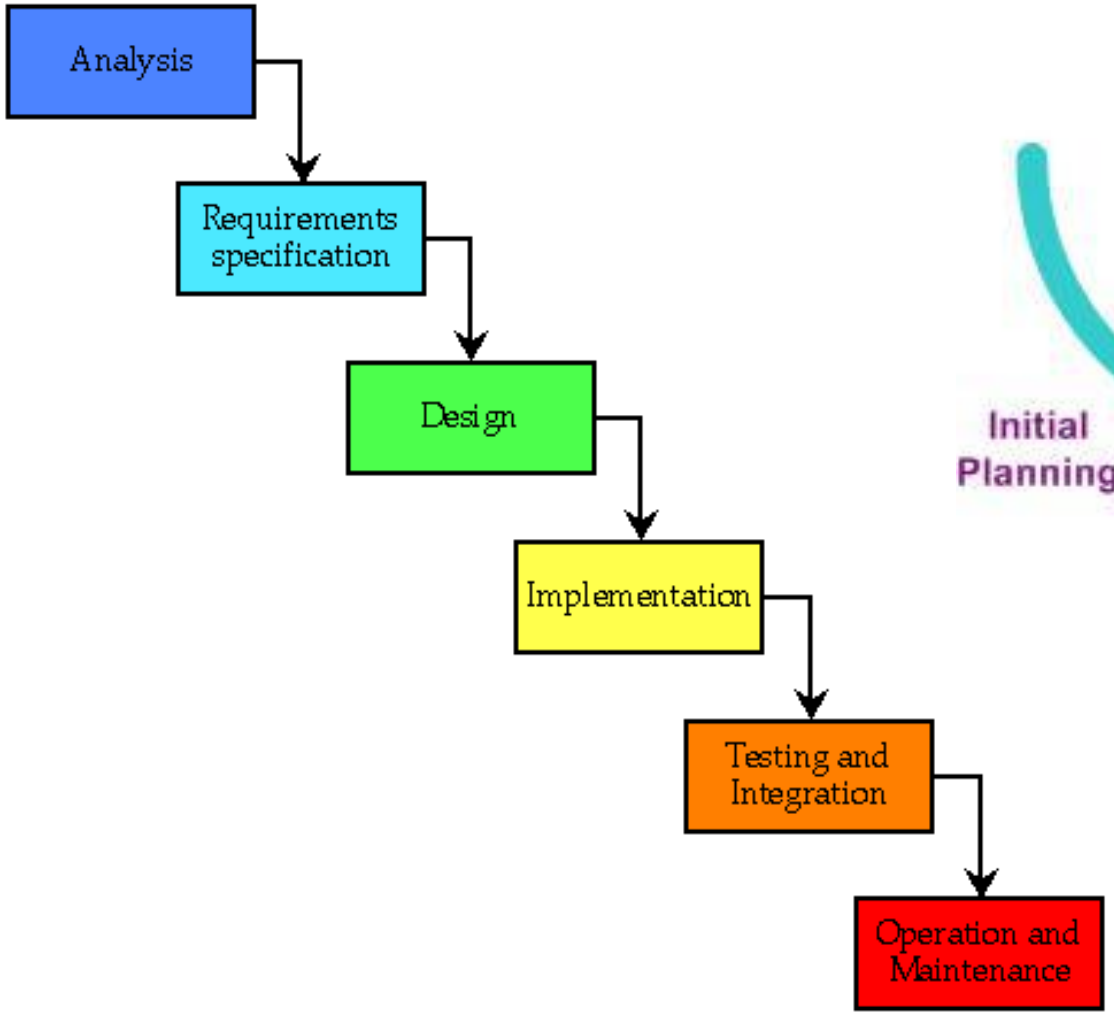
on-line access from the Library website

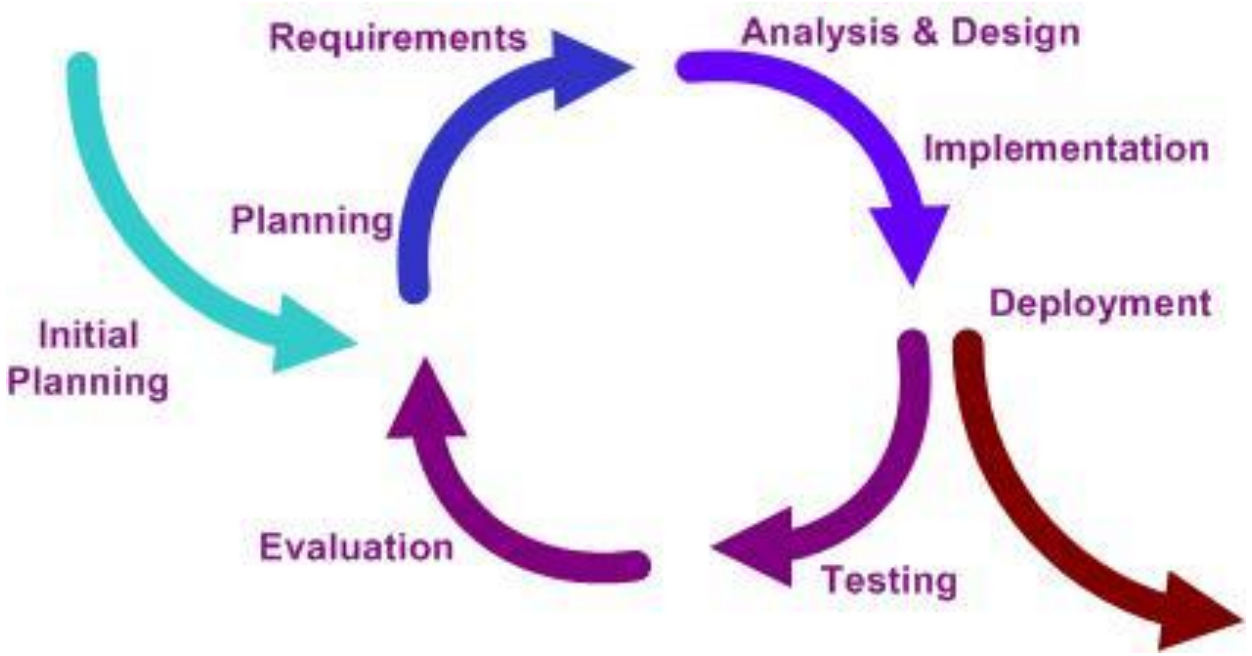


Systems Analysis and Design for E-business

Approaches to Systems Development

Waterfall model vs iterative and incremental frameworks





Prof Tamas Kiss

Systems Analysis and Design for E-business

Approaches to Systems Development

Structured - "Hard" methodologies

- Linear development, waterfall model
- Well defined stages and order
- E.g. **SSADM** – Structured Systems Analysis and Design Method

"Soft" methodologies

- Involve end-users more closely
- Follow an investigative rather than a technical approach
- E.g. **Checkland's soft approach** – rich pictures, CATWOEs, root definition etc.

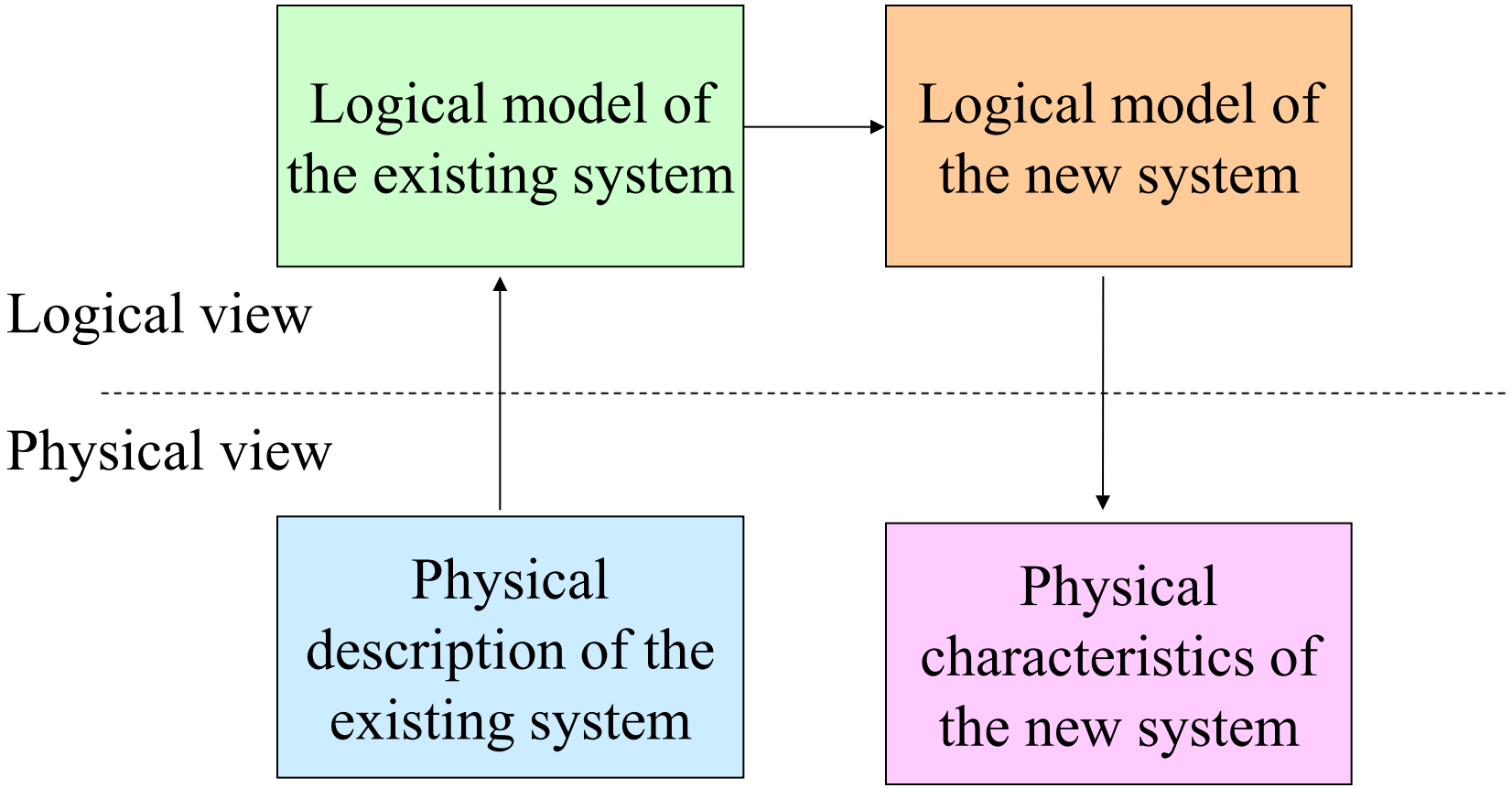
Iterative and incremental frameworks

- Based on prototyping and multiple cycles
  - **RUP** – Rational Unified Process
  - **USDP** – Unified Software Development Process
    - UML - based (Unified Modelling Language), use case driven

Prof Tamas Kiss

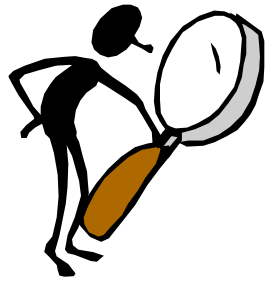
Structured - hard approaches

Logical and physical view of the system



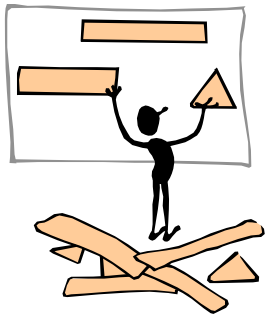
Structured - hard approaches

SDLC : Systems Development Life Cycle

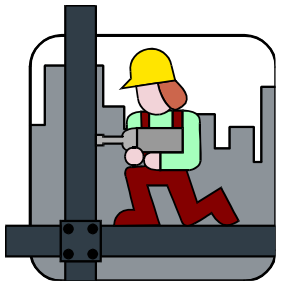


Stages:

- Investigating the existing system



- Analysing the existing system & designing the new system



- Implementing the solution



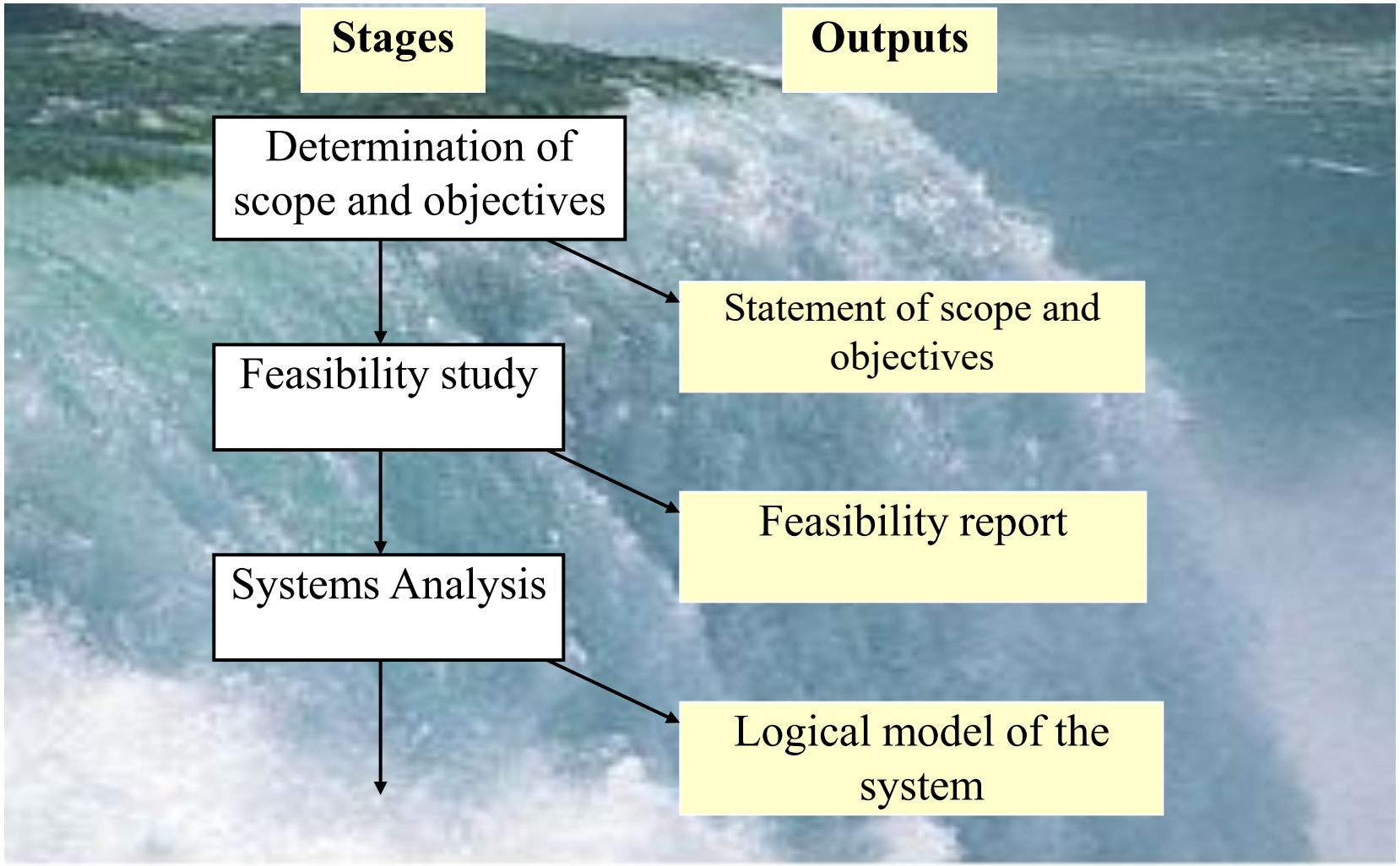
- Evaluating its effectiveness

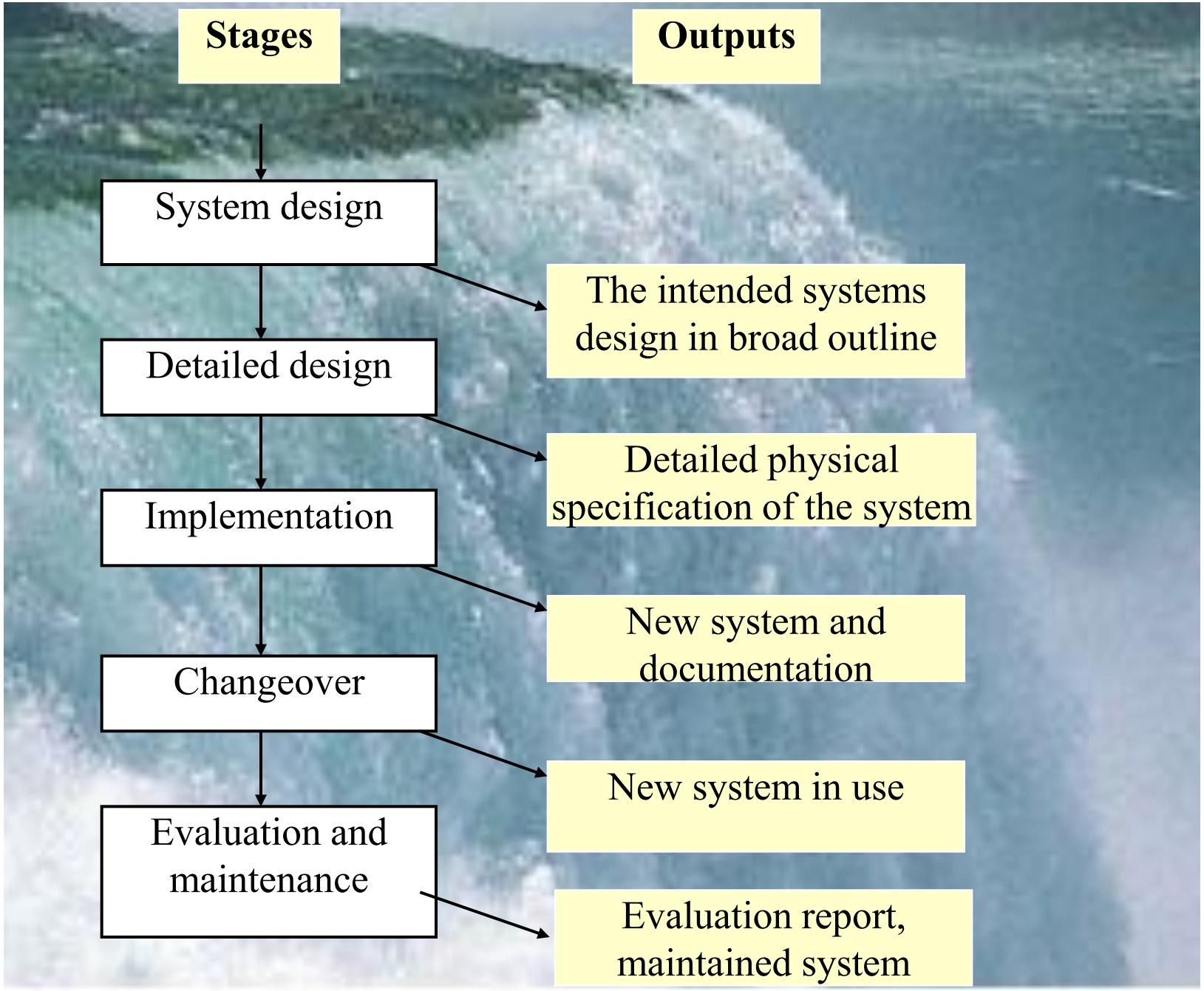
Prof Tamas Kiss

Structured - hard approaches

SDLC : Systems Development Life Cycle

- "Waterfall" model



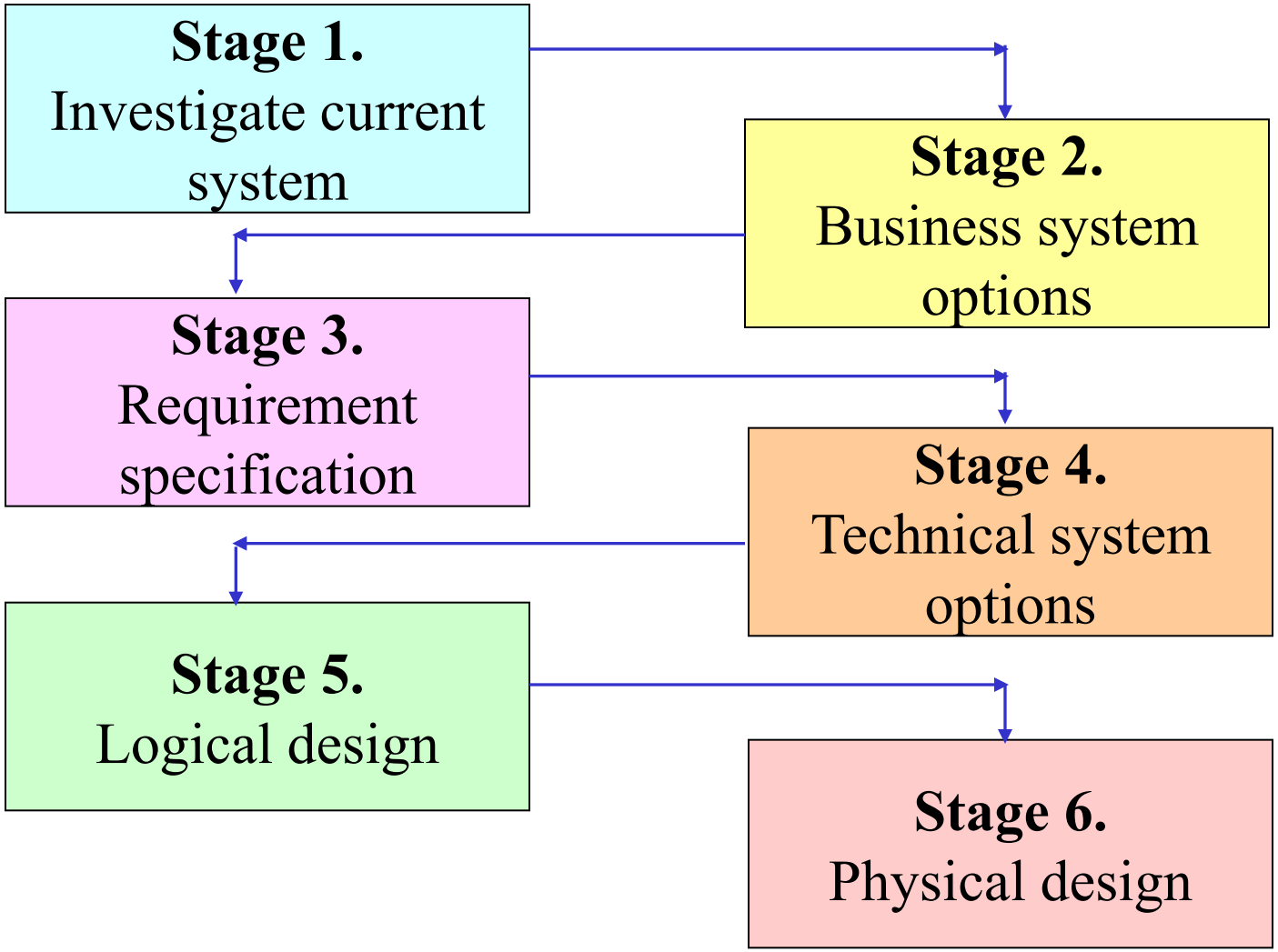


Prof Tamas Kiss

Structured - hard approaches

An Example – SSADM

(Structured System Analysis and Design Method)



Prof Tamas Kiss

Criticism of “hard” approaches

- Engineering perspective – might not be applicable
- Mathematical/logical techniques limit the range of problems
- Importance of quantitative information over qualitative
- Missing organisational context
- Possible misunderstanding between client and expert

Prof Tamas Kiss

Soft approaches

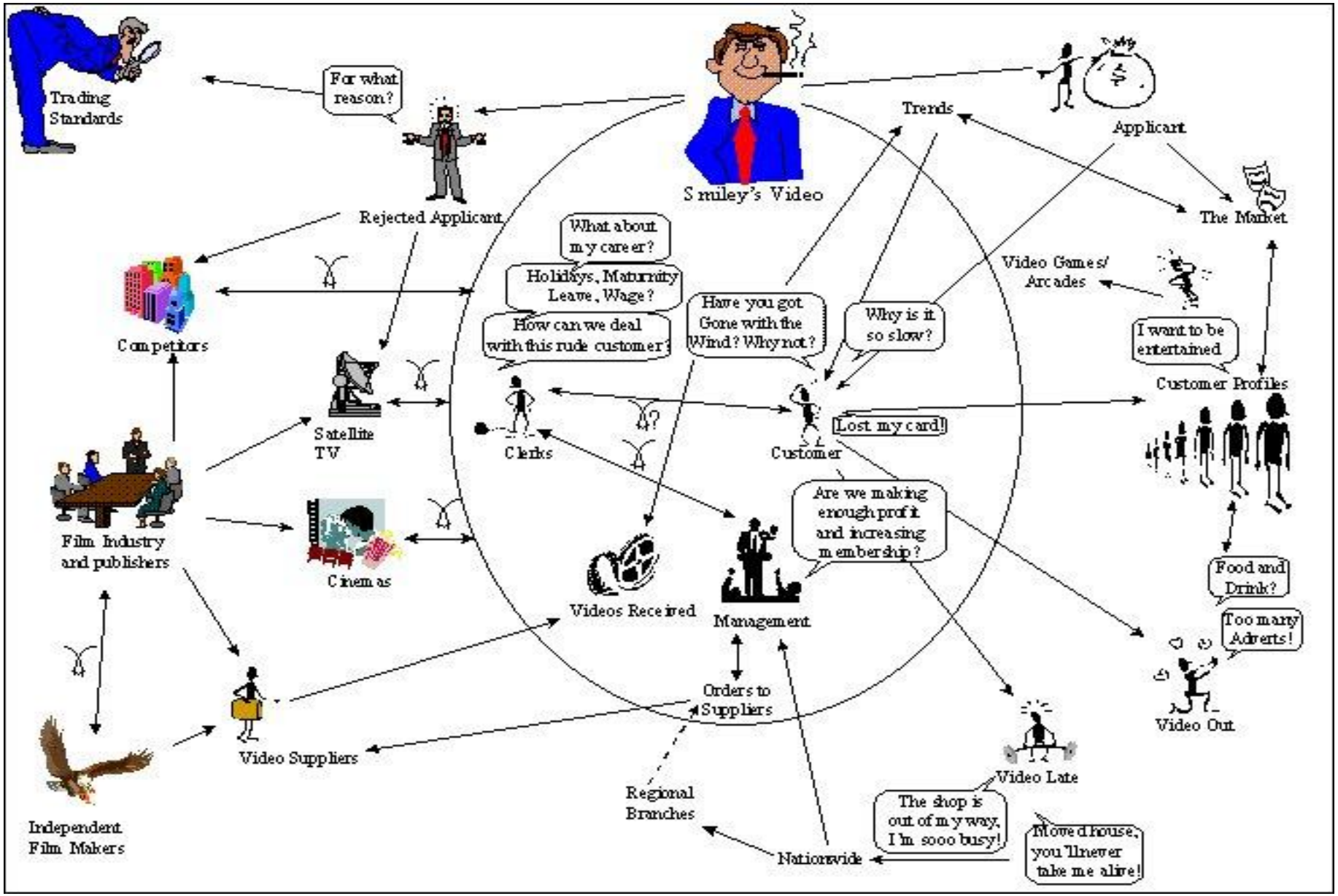
e.g. Checkland's approach

Presumptions:

- People see problems on different ways
  - e.g.: difficult exams
- People see different solutions to the same problem
  - e .g. high failure rate
    - Teacher: raising entry qualifications
    - Parents: more efficient teaching
    - Student: easier exams
- Problems are interconnected
  - solving one generates others
- The analyst's role is **to help** to understand and solve problems ("therapist")

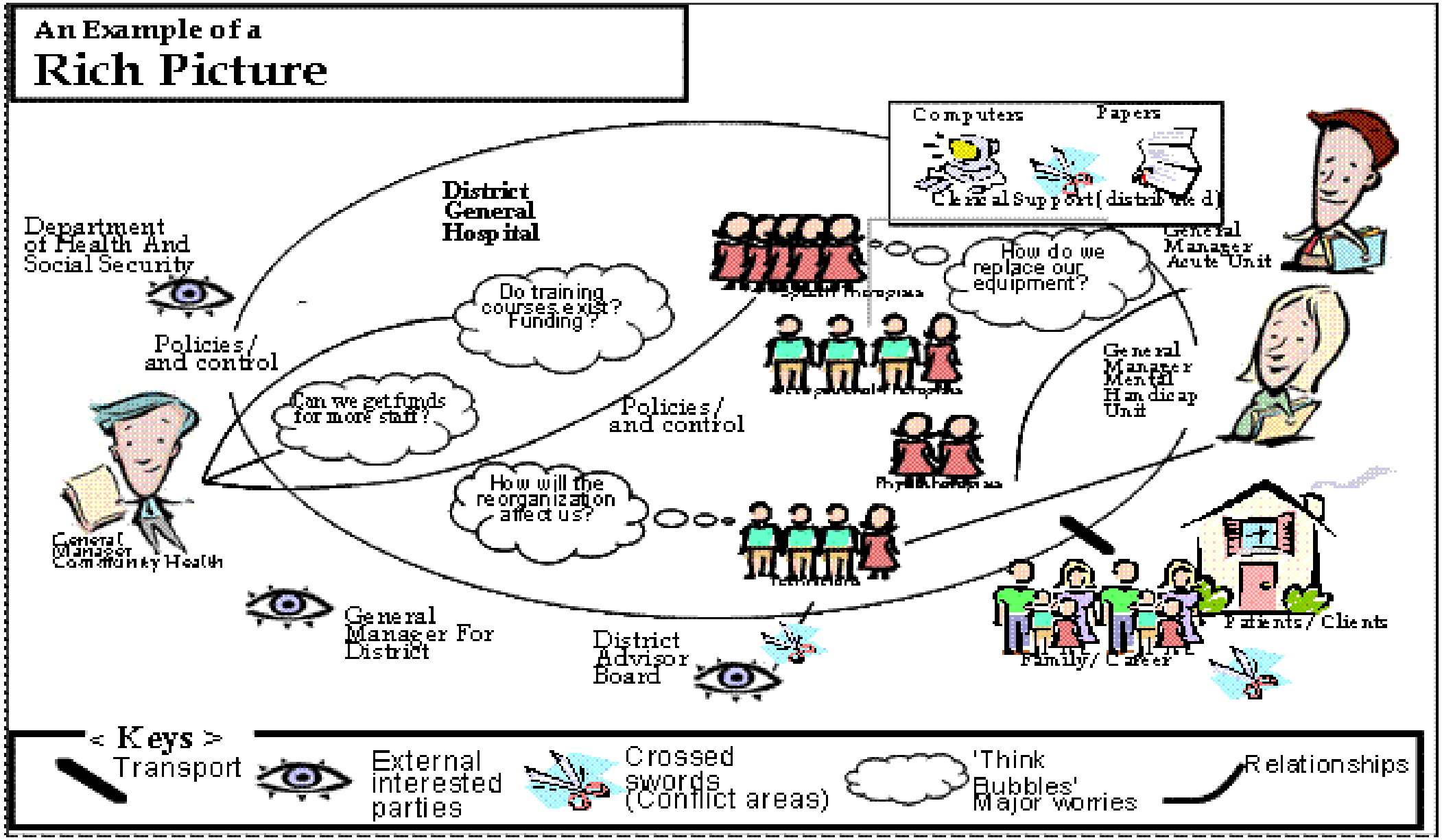
Checkland's approach

Analytical tools: e.g. Rich Pictures

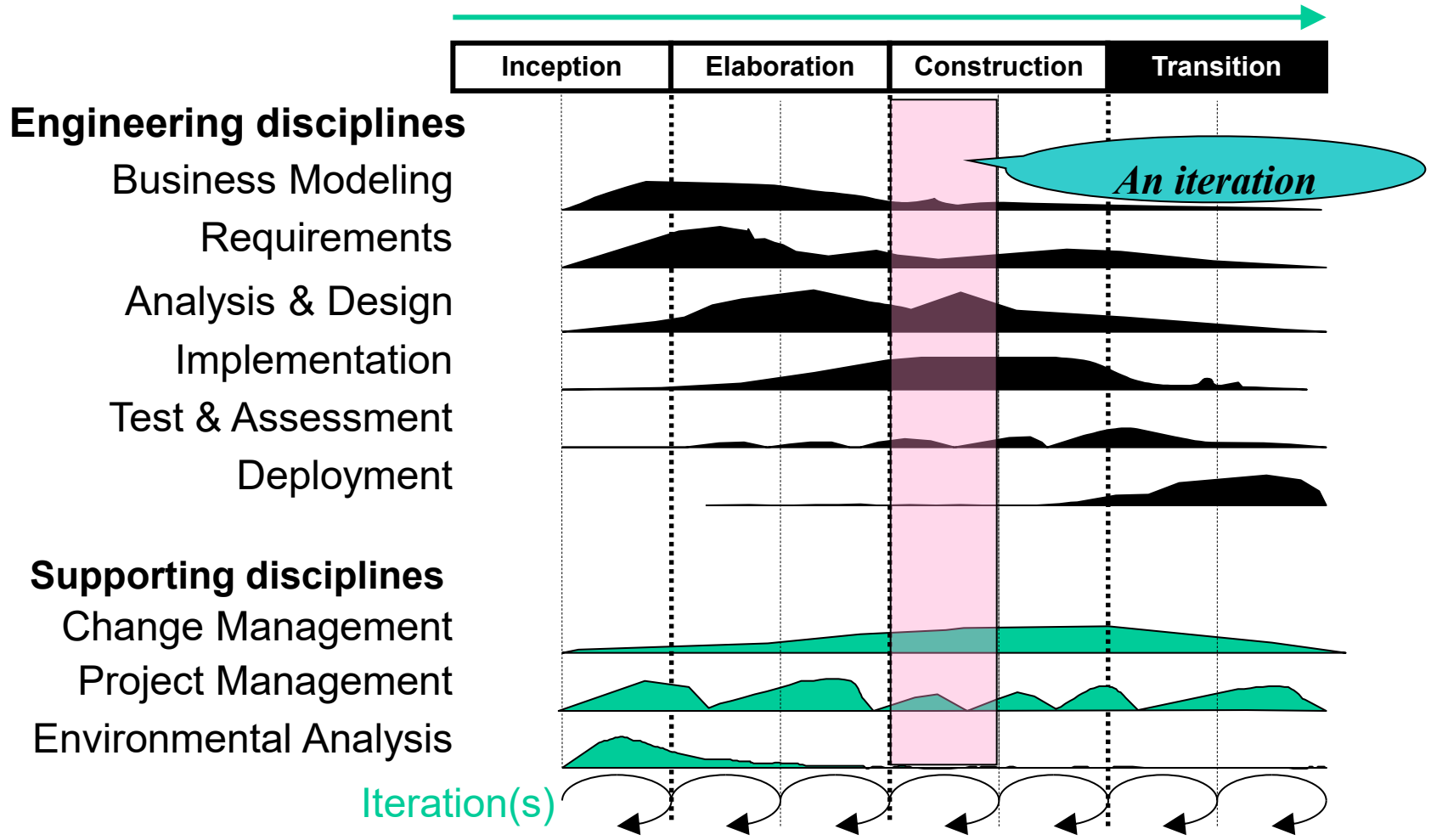


Checkland's approach

Analytical tools: e.g. Rich Pictures – a hospital

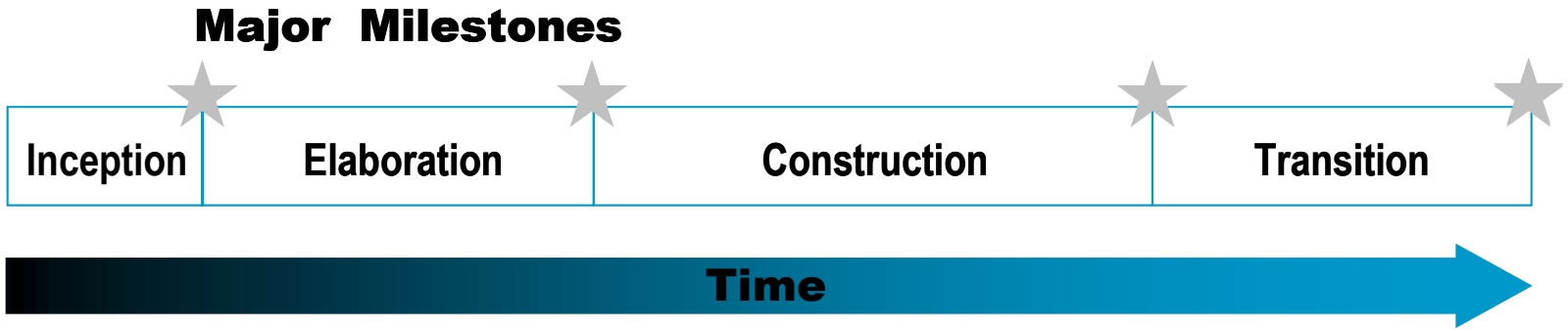


Iterative and incremental frameworks



Prof Tamas Kiss

Iterative Development Phases



Inception:

Understand what to build

- Vision, high-level requirements, business case
- Not detailed requirements

Elaboration:

Understand how to build it

- Baseline architecture, most requirements detailed
- Not detailed design

Construction:

Build the product

- Working product, system test complete

Transition:

Validate solution

- Stakeholder acceptance

Which approach/methodology to use for e-commerce/e-business systems?

E-commerce/e-business solutions:

- New developments
- Cutting edge solutions
- Very much user/use-case centred
- Dynamically changing



- **Iterative/incremental frameworks and soft methodologies may fit better than hard methodologies and linear approaches**
- Use a combination of iterative/incremental approach and soft methodology (some tools and techniques from Checkland's soft method)

Iterative frameworks – use case driven

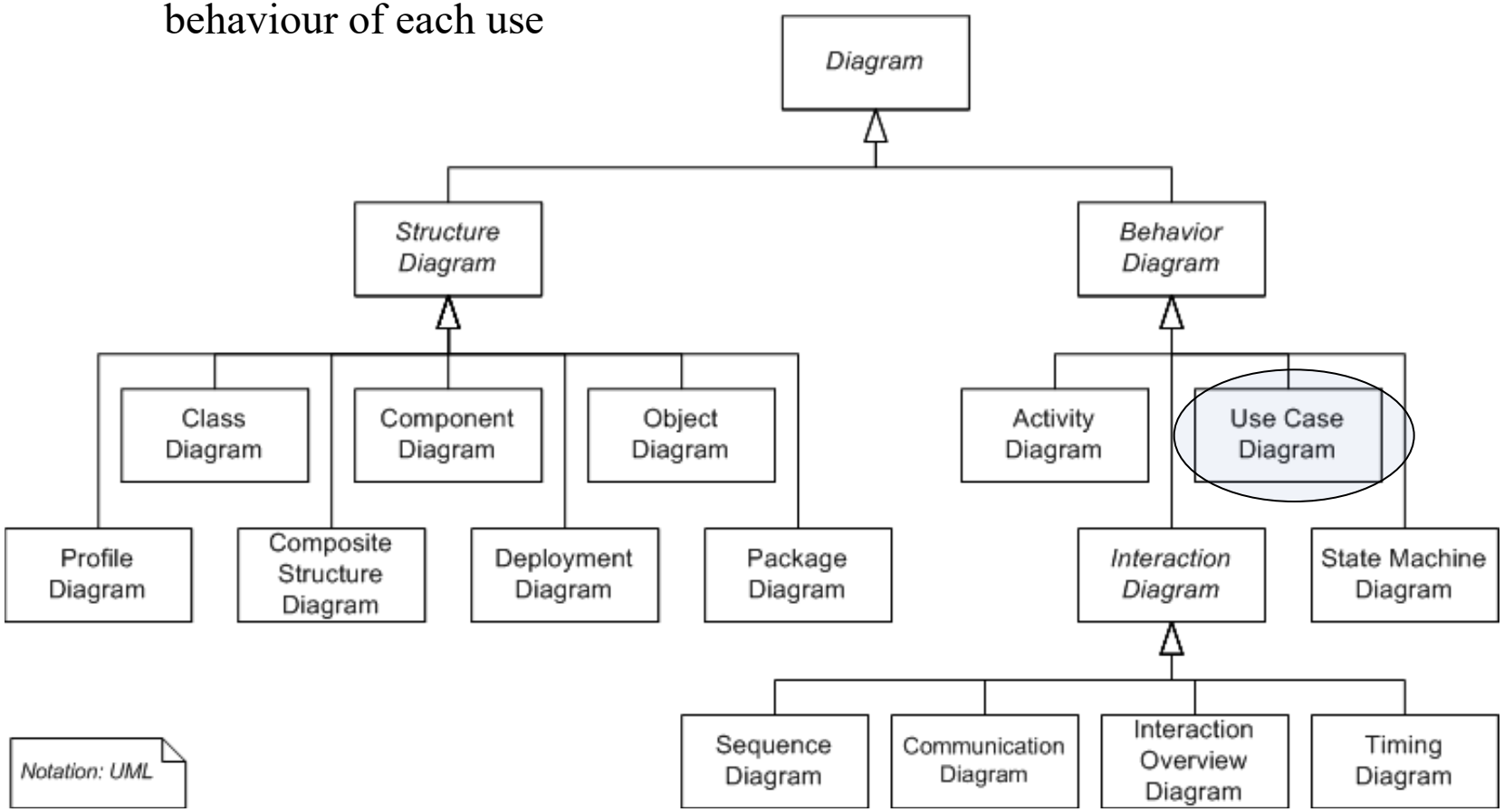
Use case diagrams

- Use cases were introduced in various previous modules
- This part of the lecture is just a brief reminder of all you learned about use-case analysis in these previous modules
- Please revise use case diagrams as necessary!

Prof Tamas Kiss

Use case diagrams

- Use cases are part of UML (Unified Modeling Language)
- A use case model generally consists of
  - One or more *use case diagrams*
  - Associated *use case descriptions* (or other diagrams) to specify the behaviour of each use



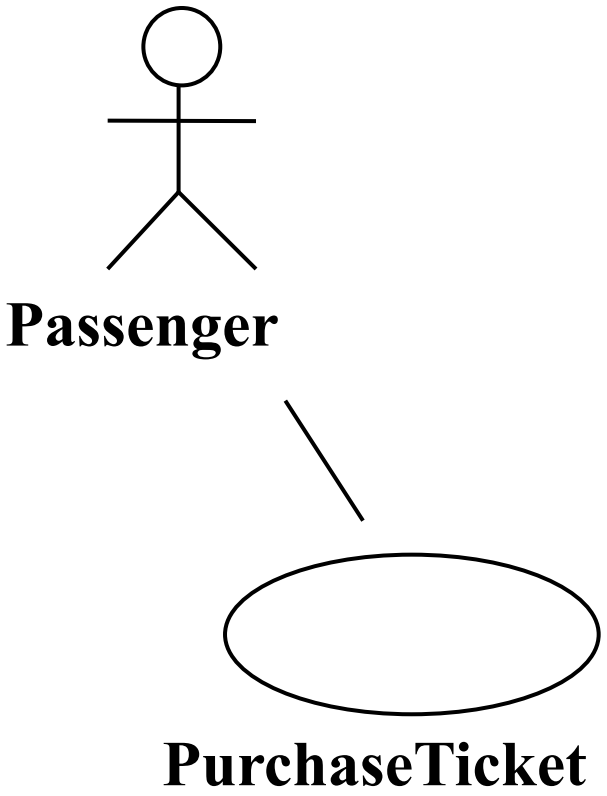
Taxonomy of UML diagrams

Use case diagrams

- Use cases form the basis of how the system interacts with the outside world (users, other systems, etc.)
- Describes WHAT the system (as a “Black Box”) does from a user’s perspective
- Use cases were designed to capture, via a combination of structured text & graphics, the functional requirements of a system
- Use cases are usually described in a textual document that accompanies a use case diagram; the combination of these use case diagrams & their supporting documentation is known as a use case model
- Illustrate a system’s intended functions (use cases), its surroundings (actors) & the relationships between them

Prof Tamas Kiss

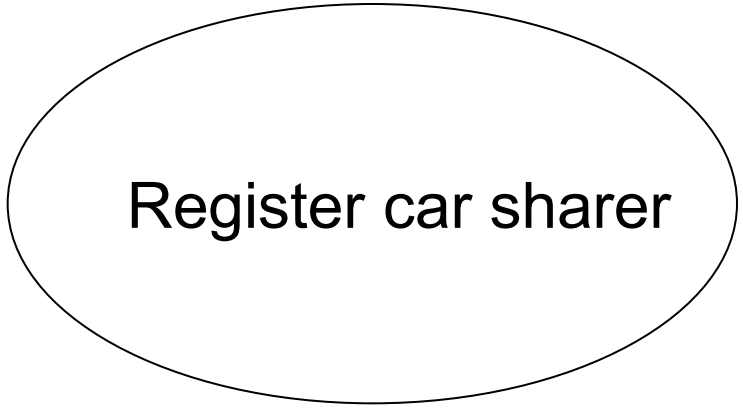
Use Case Diagrams



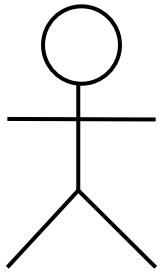
- ***Actors*** represent roles, that is, a type of user of the system
- ***Use cases*** represent a sequence of interaction for a type of functionality
- The ***use case model*** is the set of all use cases. It is a complete description of the functionality of the system and its environment

Use Cases

- A ***use case*** describes a sequence of actions that the system performs for the actors so that they can achieve what they want to achieve (an observable result of value)
- *use case* drawn as ellipse
- *use case* name drawn in or beneath the ellipse
- *use case* name is text string of letters, numbers, punctuation (except colon)
- *use case* name is:
  - active verb (upper case initial letter) followed by noun phrase
  - E.g.: Borrow book, Purchase ticket, Put into basket, etc.



Actors

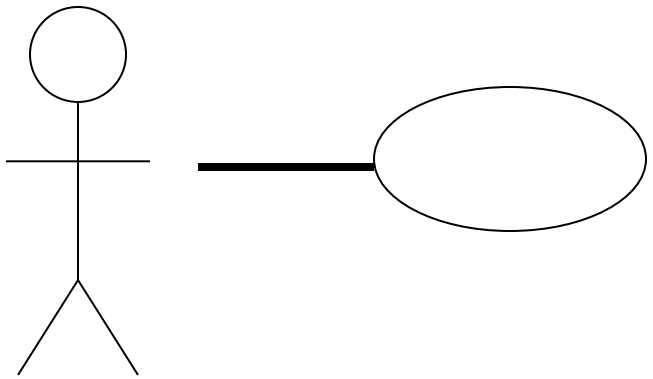


Passenger

- Actors represent the ***roles*** that people, other systems or devices take when communicating with a particular use case or use cases
  - an *actor* is a person or system that interacts with a use case
  - *actors* are generally the users of the system being modelled
  - the *actors* generally define the *boundary* of the system
- An actor is represented as a stick person with the name of the role underneath; it is the ***role*** that is being represented
- Actors, for example: Buyer, LibraryMember, Passenger, etc.

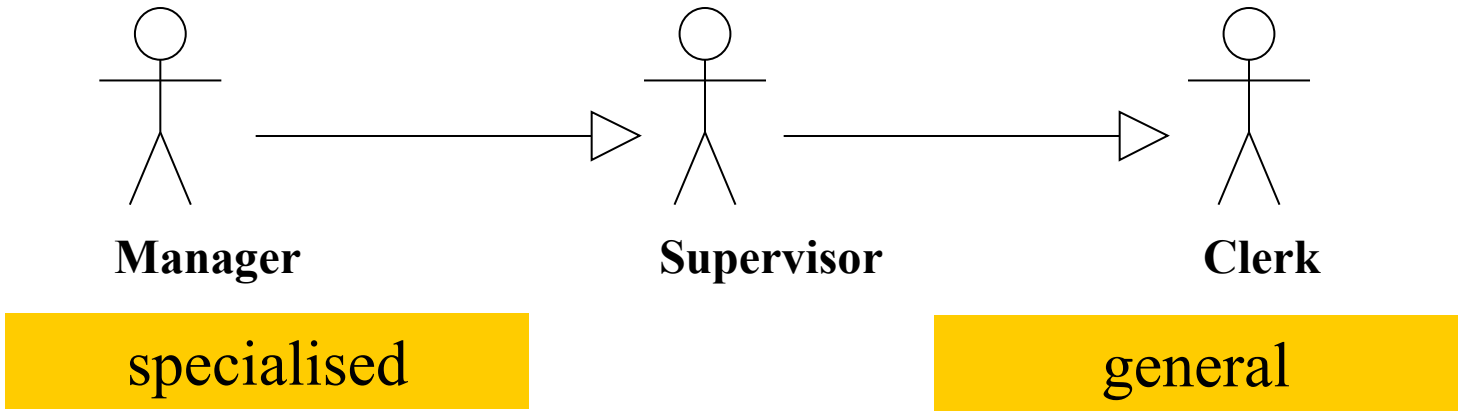
Relationship between actors and use cases

- a line is drawn between the actor and the use case to represent a relationship
- a single actor may be associated with more than one use case
- a single use case may be associated with more than one actor



Generalisation between Actors

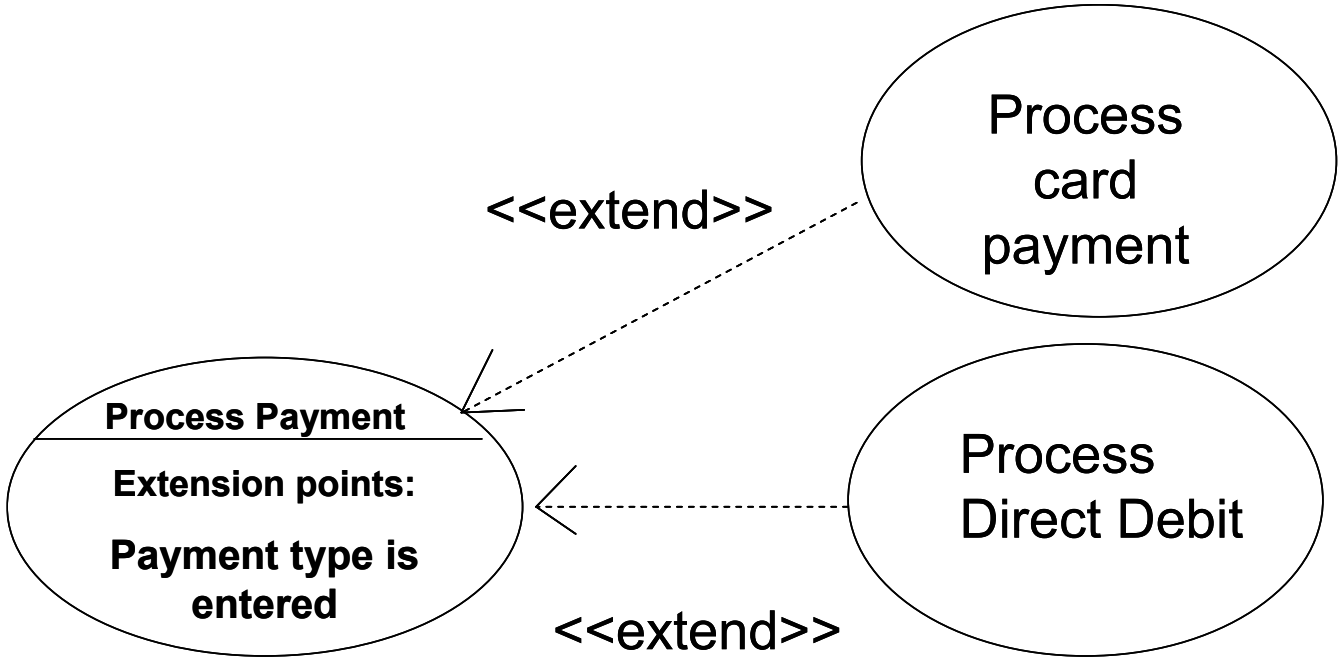
- When several actors as part of their roles, also play a more generalized role, it is described as **generalization**
- The behavior of the general role is described in an actor super-class
- The specialized actors inherit the behavior of the super-class and extend it in some way
- Relationships between actors are not always necessary



Prof Tamas Kiss

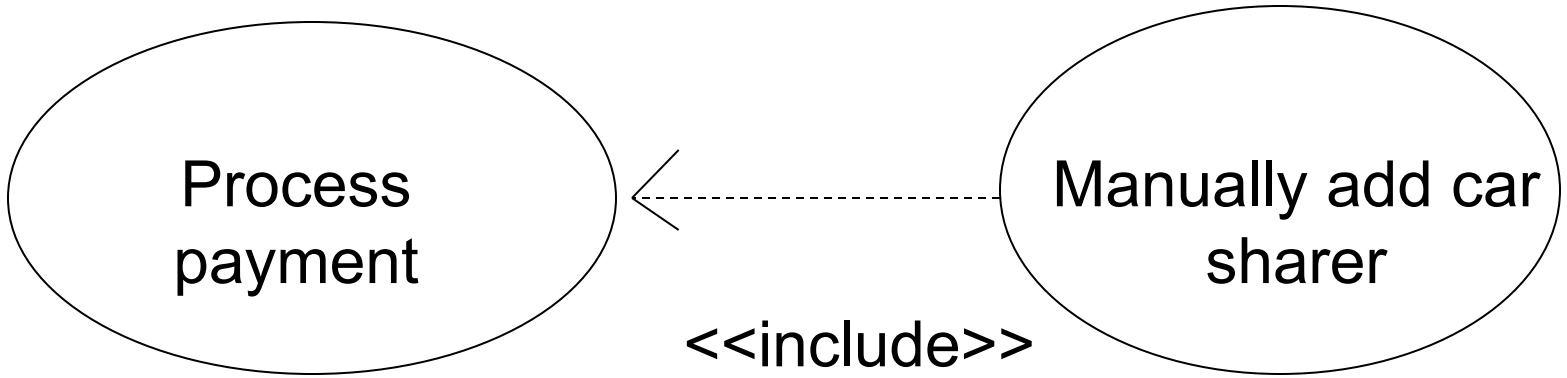
The <<*extend*>> *r*elationship between use cases

- Sometimes a particular use case has an extra set of actions when required, so the use case can be extended by another use case when a particular condition is met (or in exceptional cases)
- <<*extend*>> relationships represent exceptional or seldom invoked cases.
- The direction of an <<*extend*>> relationship is towards the extended use case



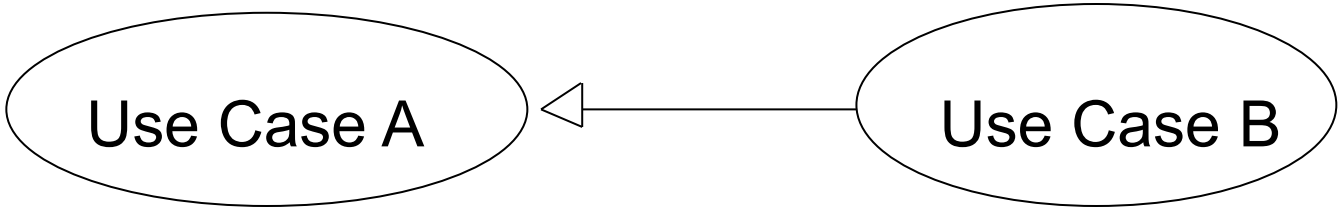
The <<*include*>> *r*elationship between use cases

- a use case can include all the behaviour of another use case
- the included use case is performed every time when the including use case is performed.
- The direction of an <<*include*>> relationship is from the including use case towards the included use-case
- **Example:** "Processing payment" exists as a separate use case but it is performed whenever a new car sharer is added manually.

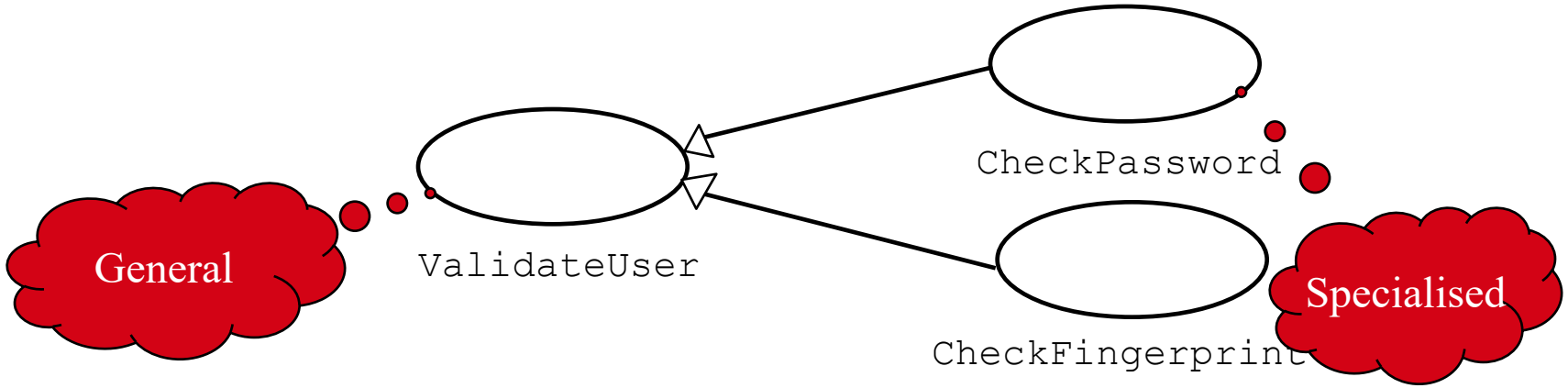


Generalization relationship between use cases

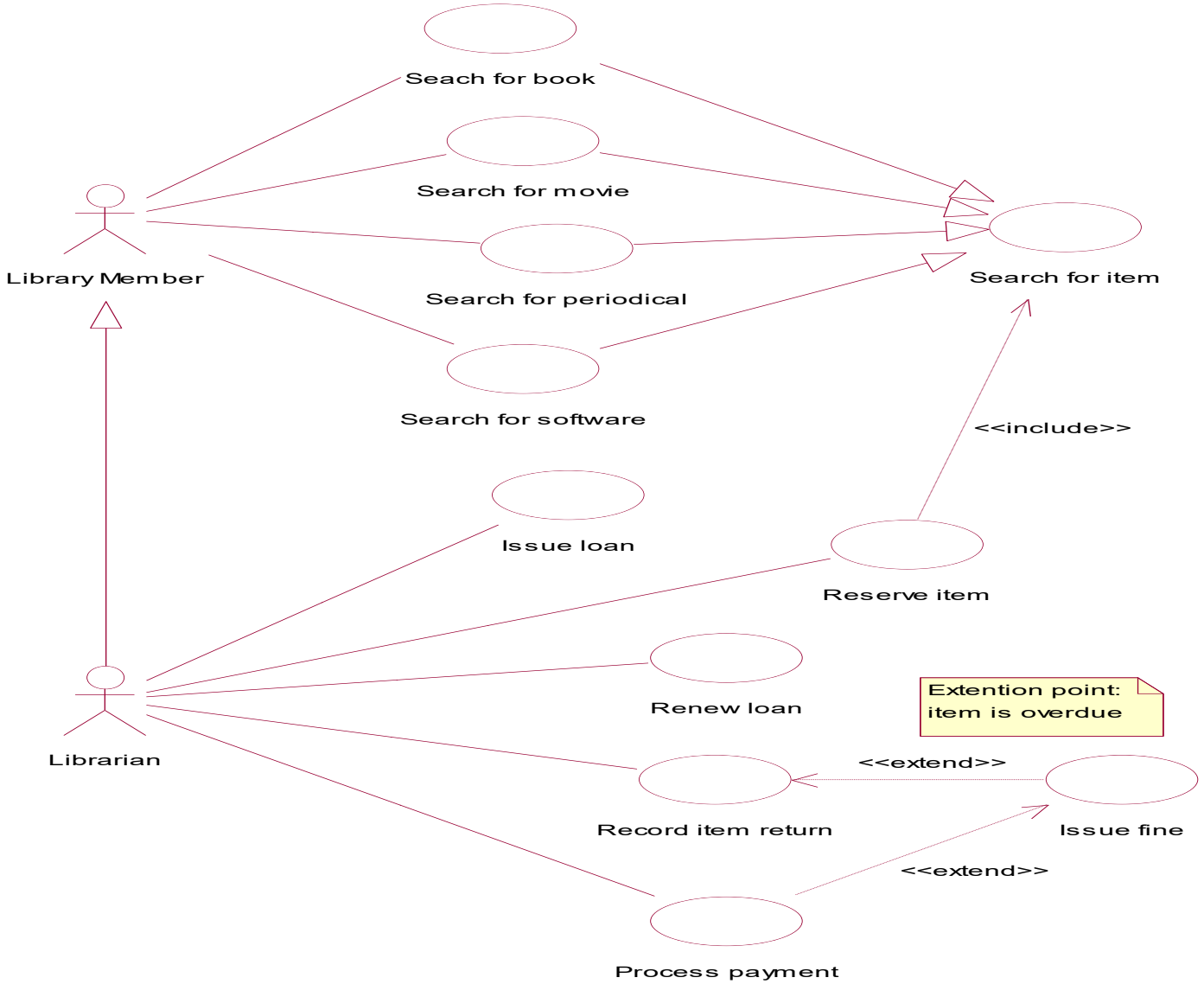
- Use Case B has all the actions of Use Case A, plus some other things.
- *Use Case B is more specialised than Use Case A*
- *Use Case A is more general than Use Case B*



- Example:
  - Consider the use case “ValidateUser”, responsible for verifying the identity of the user. The customer might require two realizations: “CheckPassword” and “CheckFingerprint”



Use Case Diagram Example – A library



Use Case Description

- A use case can be described by text.
- This text can be free form or structured.
- There are various ways of structuring a use case description.
- The approach adopted is decided by the analyst or specified in advance by the practice of the software developers or the rules of the particular project.

Use case Description Example – Issue Loan

Use Case Name	Issue loan	
Brief Description	A member comes to the counter with the item(s) that they picked from the shelve. They hand over the item(s) that they would like to borrow together with their library card. The librarian checks the member account using their member ID to see if they are allowed to borrow the items. If the member can, the librarian records the item(s) as new loans against the member's account and hands them back to the member.	
Actors	Librarian	
Purpose	To allow members to borrow books. This use case enables library to maintain a loan history for members which in turns helps to keep track of item i.e. where an item currently is, whether it is available in the library, or it is due or it is overdue and so forth.	
Pre-conditions	•Member has not reached the loan limit which is currently 10 items at any particular time •Member has no outstanding fines •Member has no overdue items •Item is not a reference only (not mentioned in the interview)	
Main Flow	**Actor Action** **1. Input item id** 3. Input member's id 5. --- 7. --- 9. Confirm loan request on item	**System Response** 2. Check item's loan type 4. Check member's outstanding fines. 6. Check member's number of current loans. 8. Check overdue loans 10. Confirm successful loan
Alternative Courses	In step 2, system rejects the loan request because the item is only in-library use. In step 4, system rejects the loan request because the member has outstanding fines from current or previous loans. In step 6, system rejects the loan request because the member has already reached the loan limit. In step 8, system rejects the loan request because one or more of the member's current loans are overdue.	
Post-conditions	The items status is updated 'on loan' The member's current number of loans increased by 1 The system goes back to the stage where actor can issue a new loan.	

Use Case Diagrams: Summary

- Use case diagrams represent external behavior
- Use case diagrams are useful as an index into the use cases
- Use case descriptions provide meat of model, not the use case diagrams.
- All use cases need to be described for the model to be useful.

Prof Tamas Kiss

Use Cases vs. Requirements

- A requirements document states what the system shall do. Use Cases describe actions that the users take and the responses that the system generates
- Use Cases are often used as a means to elicit requirements
- Requirements may be effectively documented as Use Cases
  - Better traceability
  - Easier user validation of functional requirements
  - Helps structuring the users manuals
  - A tool for finding classes
- Use-Cases **are not** well suited for capturing non-functional requirements.

Mapping requirements to use cases

An example:

a system to support the administration of final year projects

User Requirements		***Use Case Mapping***
1.	Students shall be able to view details of academics who will be supervising projects in the academic year.	View supervisor list
2.	Students *should* be able to view the details of the projects topics that are on offer by the project supervisors.	View projects on offer
3.	Supervisors and moderators shall be able to record their marks when they have finished marking the project submissions that they have been allocated to.	Record mark awarded
4.	Supervisors *should* be able to add/modify the details of the students and the projects that they have agreed to supervise.	Add/modify supervised project
5.	Supervisors *should* be able to set up a project meeting for their own students.	Set up project meeting
6	Supervisors shall be able to keep a log of all meetings that they have arranged for their project students.	Log details of project meeting
7.	Supervisors shall be able to print a copy of or the meeting logs or email them to the relevant student.	Print/email meeting log
8.	Supervisors *should* be able to put up/modify the details of the projects that they are offering for students to take on.	Offer project topic
9.	Supervisors *should* be able to view the details of the students and the projects they are supervising.	Search student

Prof Tamas Kiss

Any questions?

